ÉCOLE SUPÉRIEURE PRIVÉE D'INGÉNIERIE ET DE TECHNOLOGIES (ESPRIT)
Unité Pédagogique : Architecture des Systèmes d'Information (UP ASI)

# ATELIER 3 : SPRING DATA JPA

## Interfaces Repository, Opérations CRUD et Analyse Qualité SonarQube

| | |
|---|---|
| **Projet :** AutoLoc API (Microservice de Gestion de Location de Véhicules) | **Niveau :** ASI 26-27 |
| **Technologies :** Java 17+, Spring Boot 3, Spring Data JPA, Hibernate, MySQL, Lombok | **Outils :** IntelliJ IDEA, SonarQube for IDE (SonarLint) |
| **Package analysé :** tn.esprit.autoloc.repository | **Auteur :** Rapport TP Réalisé |

## 1. Objectifs Pédagogiques & Contexte de l'Atelier

Dans le cadre du développement de l'application **AutoLoc**, l'Atelier 2 a permis de modéliser les neuf entités du domaine métier avec leurs relations JPA (OneToOne, OneToMany, ManyToOne, ManyToMany). L'objectif de cet **Atelier 3** est de mettre en place la **couche Repository** afin d'assurer la persistance et la manipulation de ces entités en base de données MySQL, tout en appliquant les meilleures pratiques de développement et d'analyse statique.

**Les principaux axes traités dans cet atelier sont :**

- **Compréhension de Spring Data JPA :** Architecture de la couche de persistance et génération dynamique de proxies.
- **Hiérarchie des interfaces :** Rôle précis de Repository, CrudRepository, ListCrudRepository, PagingAndSortingRepository et JpaRepository.
- **Implémentation des 9 Repositories :** Respect strict du package tn.esprit.autoloc.repository et de la convention I...Repository.
- **Maîtrise du cycle de vie CRUD :** Fonctionnement de save() (persist vs merge), findById() (Optional) et deleteById().
- **Contrôle Qualité & Clean Code :** Analyse et correction d'anomalies critiques via **SonarQube for IDE (SonarLint)**.

## 2. Rappels Théoriques Approfondis : De JPA à Spring Data JPA

Pour bien appréhender Spring Data JPA, il est fondamental de distinguer les trois niveaux de la pile de persistance Java :

| Élément | Nature | Rôle & Responsabilité |
|---|---|---|
| **JPA (jakarta.persistence)** | Spécification standard (JSR) | Définit les annotations fondamentales (@Entity, @Id, @OneToMany, etc.) et l'interface standard EntityManager. N'exécute aucun code par lui-même. |
| **Hibernate** | Implémentation ORM | Implémente la spécification JPA. Traduit les objets Java en requêtes SQL adaptées au SGBD (MySQL), gère le contexte de persistance (cache L1), le *Dirty Checking* et les transactions. |

| | | |
|---|---|---|
| **Spring Data JPA** | Abstraction Spring | Couche de haut niveau au-dessus de JPA/Hibernate. À partir d'une simple déclaration d'interface, Spring génère à l'exécution un **proxy dynamique** implémentant toutes les requêtes CRUD. **Supprime totalement l'écriture manuelle de classes DAO / RepositoryImpl.** |

## 2.1. Hiérarchie des Interfaces Spring Data JPA

Spring Data propose une hiérarchie modulaire d'interfaces génériques paramétrées par <T, ID> (où T est la classe de l'entité et ID le type wrapper de sa clé primaire, ex: Long) :

```
Repository                    (Interface marqueur vide, aucun contrat)
■■■ CrudRepository              (Opérations CRUD de base, renvoie Iterable)
■   ■■■ ListCrudRepository      (CRUD avec méthodes retournant des List)
■■■ PagingAndSortingRepository  (Tri & Pagination, renvoie Iterable)
    ■■■ ListPagingAndSortingRepository (Tri & Pagination renvoyant des List)

JpaRepository = ListCrudRepository + ListPagingAndSortingRepository
                     + QueryByExampleExecutor + Méthodes JPA (flush, saveAndFlush, batch)
```

**Évolution majeure Spring Data 3 / Spring Boot 3 :**
Historiquement, PagingAndSortingRepository héritait de CrudRepository. Depuis Spring Data 3, ces deux branches sont **séparées** pour permettre une meilleure modularité. L'interface JpaRepository combine désormais ListCrudRepository et ListPagingAndSortingRepository, ce qui en fait l'interface la plus polyvalente et la plus adaptée pour les applications d'entreprise comme AutoLoc.

## 2.2. Mécanisme Détaillé des Opérations CRUD et Cycle de Vie JPA

L'utilisation correcte de Spring Data JPA nécessite de comprendre ce qui se passe sous le capot lors de chaque appel de méthode :

| Méthode | Rôle & Type Retour | Comportement Hibernate & SQL sous-jacent |
|---|---|---|
| **save(entity)** | <S extends T> S | • **Si id == null (nouvelle entité) :** Déclenche persist(). Exécute un INSERT SQL. MySQL génère la clé primaire auto-incrémentée qui est injectée dans l'objet retourné.<br>• **Si id != null (entité existante) :** Déclenche merge(). Exécute un SELECT pour charger l'état en base, applique le Dirty Checking, puis émet un UPDATE SQL si nécessaire.<br>**Règle d'or :** Toujours récupérer la variable de retour : contrat = contratRepo.save(contrat); |
| **findById(id)** | Optional<T> | Effectue un SELECT ... WHERE id = ?. L'utilisation de Optional évite les NullPointerException et s'intègre parfaitement avec .orElseThrow(() -> new ResourceNotFoundException(...)) dans la couche Service. |
| **deleteById(id)** | void | Charge d'abord l'entité dans le contexte de persistance (connaissance de son état), puis exécute le DELETE. **Avantage crucial :** Déclenche les mécanismes de cascade = CascadeType.REMOVE et orphanRemoval = true. |
| **saveAndFlush(entity)** | <S extends T> S | Effectue un save() suivi immédiatement d'un flush(). Force la synchronisation immédiate du cache Hibernate vers la base de données sans attendre la fin du commit de transaction. |
| **deleteAllInBatch()** | void | **ATTENTION :** Exécute un DELETE FROM table direct en une seule requête SQL. **Contourne complètement le contexte de persistance :** Aucune cascade ni suppression orpheline n'est appliquée. Risque d'échec par violation de contrainte d'intégrité (Foreign Key) si des entités enfants existent (ex: paiements liés à un contrat). |

# 3. Mise en Œuvre dans le Projet AutoLoc API

Conformément aux directives de l'atelier, neuf interfaces repository ont été créées dans le package tn.esprit.autoloc.repository. Chaque interface hérite de JpaRepository<Entite, Long> et respecte la convention de nommage avec le préfixe I.

| Entité Métier (domain) | Interface Repository (repository) | Interface Étendue & Justification Technique |
|---|---|---|
| **Agence** | IAgenceRepository | JpaRepository<Agence, Long><br>Gestion complète des agences, pagination pour la liste d'administration, requêtes de recherche par ville. |
| **Employe** | IEmployeRepository | JpaRepository<Employe, Long><br>Listing sous forme de List<Employe>, gestion des rôles (Agent, Gestionnaire) par agence. |
| **Vehicule** | IVehiculeRepository | JpaRepository<Vehicule, Long><br>Gestion du parc automobile, pagination du catalogue public, tri par tarif journalier (Sort). |
| **Equipement** | IEquipementRepository | JpaRepository<Equipement, Long><br>Catalogue des options/équipements (GPS, siège bébé). Retour direct en List. |

| | | |
|---|---|---|
| **Client** | IClientRepository | JpaRepository<Client, Long><br>Enregistrement des clients, recherche par email unique, pagination de l'annuaire client. |
| **Reservation** | IReservationRepository | JpaRepository<Reservation, Long><br>Suivi des plannings de réservation, filtrage par statut et date, manipulation sous forme de List. |
| **Contrat**<br>***(Entité Pilote)*** | IContratRepository | JpaRepository<Contrat, Long><br>Gère la génération des contrats. Accès aux méthodes saveAndFlush() et propagation de la cascade vers les paiements. |
| **Paiement** | IPaiementRepository | JpaRepository<Paiement, Long><br>Lecture et audit comptable direct des transactions financières. *(Les écritures/suppressions restent orchestrées par le Contrat parent)*. |
| **Maintenance** | IMaintenanceRepository | JpaRepository<Maintenance, Long><br>Historique et suivi des entretiens périodiques et réparations techniques des véhicules. |

# 4. Analyse Qualité de Code avec SonarQube for IDE (SonarLint)

L'intégration d'un outil d'analyse statique comme **SonarQube for IDE (SonarLint)** dans l'environnement de développement permet de détecter les mauvaises pratiques (code smells), les bugs potentiels et les failles de sécurité dès la phase d'écriture du code.

**Tableau des anomalies relevées, règles associées et corrections apportées :**

| # | Anomalie Détectée | Règle SonarLint & Justification Technique | Correction Appliquée |
|---|---|---|---|
| 1 | **Usage de l'annotation @Data sur les entités JPA** | **Règle java:S2160 / Danger Lombok JPA :** @Data génère automatiquement un equals(), hashCode() et toString() incluant tous les champs.<br>• **Boucle infinie :** Sur les relations bidirectionnelles (ex: Contrat <-> Paiement), l'appel à toString() provoque un **StackOverflowError**.<br>• **Instabilité des Sets :** Si la clé primaire est générée lors du persist, le hashCode() change, cassant l'intégrité des collections HashSet/Set. | Remplacement systématique de @Data par des annotations explicites : @Getter, @Setter, @NoArgsConstructor, @AllArgsConstructor. |
| 2 | **Imports inutilisés et imports Wildcard** | **Règle java:S1128 (Unused imports) :** La présence d'imports superflus ou de wildcards globaux (import java.util.*;) alourdit le code, peut masquer des conflits de noms de classes et dégrade la maintenabilité globale. | Suppression de tous les imports inutiles et réorganisation des imports sous forme explicite et ciblée. |
| 3 | **Non-respect du type d'identifiant générique** | **Règles de généricité Java & Spécification Spring Data :** L'utilisation d'un type primitif (ex: long au lieu de Long) dans les paramètres de type générique d'une interface provoque des erreurs ou warnings de compilation. De plus, les conventions de nommage imposent le préfixe I pour les interfaces de cet atelier. | Vérification et uniformisation de toutes les interfaces sous la forme JpaRepository<Entite, Long>. |
| 4 | **Utilisation risquée des suppressions en lot (deleteInBatch)** | **Intégrité Référentielle & Cycle de vie JPA :** Les méthodes batch exécutent directement du SQL brut sans passer par l'EntityManager. Les options CascadeType.REMOVE et orphanRemoval sont ignorées, ce qui risque de laisser des enregistrements orphelins ou de lever des contraintes de clés étrangères. | Recommandation d'utiliser deleteById() dans la couche Service pour garantir l'exécution des cascades définies. |

# 5. Validation, Exécution et Interprétation des Logs

Lors du démarrage de l'application Spring Boot avec les neuf interfaces repository configurées, Spring Data analyse le classpath et initialise les proxies dynamiques :

```
[INFO] --- compiler:3.13.0:compile (default-compile) @ autoloc-api ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 24 source files with javac [debug parameters release 17] to target\classes
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[main] o.s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories
in DEFAULT mode.
[main] o.s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning in
 48 ms. Found 9 JPA repository interfaces.
```

**Constatations :**
1. **Aucune ligne DDL n'est générée en plus :** Le schéma relationnel en base de données correspond exactement à celui établi à l'Atelier 2.
2. **Détection automatique :** Le log Found 9 JPA repository interfaces confirme que les neuf interfaces ont été identifiées et associées à leurs proxys d'exécution respectifs.
3. **Livrable documentaire :** Le fichier docs/repository-notes.md a été rédigé et complété avec l'ensemble des justifications et résolutions de règles qualité.

## 6. Synthèse des Livrables Attendus

| Livrable | Emplacement dans le projet | Statut |
|---|---|---|
| **9 Interfaces Repository** | src/main/java/tn/esprit/autoloc/repository/I*Repository.java | **CONFORME (9/9 créées & testées)** |
| **Note Documentaire Markdown** | docs/repository-notes.md | **CONFORME (Tableaux & Justifications)** |
| **Rapport Complet PDF** | Atelier_3_Spring_Data_JPA_Rapport.pdf | **GÉNÉRÉ & INTÉGRÉ** |